Lab 2 — Get Started with Azure Cloud Shell (CLI and PowerShell)

In this lab you open Azure Cloud Shell — a browser-based terminal with the Azure CLI and Azure PowerShell pre-installed and pre-authenticated — and use it to inspect your subscription, manage the `rg-az900-labs` resource group, and edit files with the built-in editor. This maps to the AZ-900 skill area **Describe Azure management and governance** (tools for managing Azure: Azure portal, Azure CLI, Azure PowerShell, Cloud Shell).

Run in Azure Cloud Shell — https://shell.azure.com

Step 1 — Open Cloud Shell and choose Bash

1. Go to **https://shell.azure.com** (or in the portal, click the **Cloud Shell icon** `>_` in the top toolbar).
2. Sign in with your Azure account if prompted.
3. When asked to choose a shell, select **Bash**.

Cloud Shell runs a small Linux container in Azure — you never install anything locally, and you are already signed in, so no `az login` is needed.

Step 2 — Complete the first-run storage setup

On first launch Cloud Shell asks how to persist your files:

1. If you see **Getting started** with a **No storage account required** option (ephemeral session), you can pick it for quick tasks — but for this course choose **Mount storage account** so your files survive between sessions.
2. Select your subscription and click **Apply** (or use **Advanced settings** to choose the region `Southeast Asia` and an existing resource group).
3. Wait for the prompt to appear, e.g. `yourname@Azure:~$`.

The mounted storage account holds a small file share (`clouddrive`) mapped into your home directory. The ephemeral "no storage" option starts faster but wipes files when the session ends.

Step 3 — Inspect your account with the Azure CLI

```
az account show
```

Displays the subscription Cloud Shell is signed into as JSON — check the `name`, `id` and `user` fields to confirm you are in the right subscription.

```
az account show --query name -o tsv
```

The `--query` flag uses JMESPath to extract a single field, and `-o tsv` strips the quotes — a pattern you will use constantly for scripting.

Step 4 — List resource groups

```
az group list -o table
```

Shows the resource groups in your subscription in a readable table — you should see `rg-az900-labs` from Lab 1 with location `southeastasia`.

The `-o table` output format is the most readable for humans; the default `json` is best for scripts.

Step 5 — Create the resource group with the CLI (idempotent)

```
az group create --name rg-az900-labs --location southeastasia --tags course=az900
```

Returns the group's JSON with `"provisioningState": "Succeeded"` — because ARM deployments are idempotent, re-running this on the existing group simply succeeds without error (and ensures the `course=az900` tag is set). If you skipped Lab 1, this creates the group fresh.

```
az group show --name rg-az900-labs -o table
```

Confirms the group exists and shows its location and status.

Step 6 — (Optional) Try interactive mode

```
az interactive
```

Launches an interactive CLI experience with auto-completion, command descriptions and examples as you type. It may take a minute to install on first run. Type `exit` to leave it and return to Bash.

Step 7 — Switch to PowerShell and run an Az cmdlet

1. In the Cloud Shell toolbar, use the shell drop-down (top-left, showing **Bash**) and select **PowerShell** (click **Confirm**). The prompt changes to `PS /home/yourname>`.

```
Get-AzResourceGroup
```

Lists the same resource groups using the Azure PowerShell `Az` module — note `rg-az900-labs` appears with `Location: southeastasia`. Azure CLI and Azure PowerShell are two clients for the same ARM API; teams pick whichever fits their scripting culture.

```
Get-AzResourceGroup -Name rg-az900-labs | Format-List
```

Shows the full detail of one group, including the `course=az900` tag.

1. Switch back to **Bash** using the same drop-down for the remaining steps.

Step 8 — Use the built-in editor

```
mkdir -p ~/az900 && cd ~/az900
code notes.txt
```

`code .` / `code <file>` opens the built-in Cloud Shell editor (you can also click the **curly-brace icon** `{}` in the toolbar). Type a line such as `AZ-900 Lab 2 - Cloud Shell works!`, then save with **Ctrl+S** and close the editor with **Ctrl+Q** (or the `...` menu > **Save** / **Close editor**).

```
cat notes.txt
```

Prints the file contents back — proof the file was written to your Cloud Shell home directory (persisted if you mounted storage in Step 2).

Step 9 — Upload and download files

1. In the Cloud Shell toolbar, click the **Manage files** icon (folder/up-down arrows) and choose **Upload**. Pick any small file from your computer — it lands in your Cloud Shell home directory (`ls ~` to verify).
2. Choose **Manage files** > **Download**, enter the path `az900/notes.txt`, and click **Download** — the browser downloads your file.

This is the simplest way to move scripts and templates between your laptop and Cloud Shell.

Clean up

Keep everything:

- `rg-az900-labs` **is reused by all later labs — do not delete it** (it is removed at the end of the course, Lab 16).
- The Cloud Shell storage account is tiny (a few GB of Standard LRS file share) and costs only cents per month; keep it for the rest of the course. If you ever want to remove it after the course, delete the resource group named like `cloud-shell-storage-southeastasia`.

Remove only the scratch file if you like:

```
rm -rf ~/az900
```

What you learned

- Cloud Shell gives you a pre-authenticated Bash or PowerShell terminal in the browser — no local install, with optional persistent storage (or an ephemeral no-storage session).

- Core Azure CLI patterns: `az account show`, `az group list/create/show`, `--query` with JMESPath and `-o table|tsv|json` output formats.
- ARM operations are idempotent — re-running `az group create` on an existing group succeeds safely.
- Azure PowerShell (`Get-AzResourceGroup`) and the Azure CLI are equivalent management tools over the same Azure Resource Manager API.
- The built-in editor (`code`) and upload/download buttons make Cloud Shell a complete lightweight management workstation.

Files in this lab folder

File	What it is
`verify-setup.sh`	Verifies the CLI, your subscription and the shared `rg-az900-labs` resource group.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.